

Stacks Labs - PoX-5 Waterfall

Security Assessment

August 11, 2026

Prepared for:
Alex Huth
Stacks Labs

Prepared by: **Anish Naik, Jaime Iglesias, Tarun Bansal, and Quan Nguyen**

Table of Contents

Table of Contents 1
Project Summary 2
Executive Summary 3
Project Goals 6
Project Targets 7
Project Coverage 8
Summary of Findings 10
Detailed Findings 11
1. Bond rollover lets a staker reclaim collateral while still earning the old bond's final-cycle reward 11
2. Reward cycle start log field uses the pre-waterfall predicate 14
3. Miner-spend estimators model block-commits with the pre-waterfall output scheme 16
4. Nakamoto block signature ordering check only validates the first signer index 18
5. PoX-5 bond-admin resolver allows the config override on the mainnet 20
A. Vulnerability Categories 22
B. Code Quality Recommendations 24
C. Fix Review Results 25
Detailed Fix Review Results 25
D. Fix Review Status Categories 27
About Trail of Bits 28
Notices and Remarks 29

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Anish Naik, Consultant
anish.naik@trailofbits.com

Jaime Iglesias, Consultant
jaime.iglesiasbotas@trailofbits.com

Tarun Bansal, Consultant
tarun.bansal@trailofbits.com

Quan Nguyen, Consultant
quan.nguyen@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
June 4, 2026	Pre-project kickoff call
June 15, 2026	Status update meeting #1
June 29, 2026	Delivery of report draft and report readout meeting
August 11, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Stacks Labs engaged Trail of Bits to review the security of proof-of-transfer version 5 (PoX-5) integration changes to their smart contracts and node client code.

Stacks is a Bitcoin layer whose proof-of-transfer mechanism anchors leader election to the Bitcoin chain and whose Nakamoto consensus finalizes Stacks blocks through stake-weighted signer voting. The PoX-5 waterfall changes, which activate at Epoch 4.0, replace the classic reward scheme with a single sBTC taproot deposit address per reward cycle that every block-commit must pay. The per-cycle sBTC recipient and the Nakamoto block-signer set are derived from a new `pox-5` boot contract, which also introduces sBTC-backed protocol bonds and STX staking with a per-cycle reward-accounting system, and the reward-cycle-start boundary is shifted accordingly.

A team of four consultants conducted the review from June 8 to June 26, 2026, for a total of six engineer-weeks of effort. Our testing efforts focused on whether the new mechanism preserves consensus safety and backward compatibility: the validation of waterfall block-commits, the derivation of the reward set and signer set, the derivation and spendability of the per-cycle sBTC recipient address, the native locking of staked STX, and the new reward accounting. With full access to the source code and documentation, we performed static and dynamic testing of the node client and boot contracts, using both automated and manual methods.

Observations and Impact

The review did not identify any high- or medium-severity issues. We observed several practices that strengthen the changes. The new behavior is consistently gated to Epoch 4.0 through activation guards and a single waterfall-boundary value that all boundary-sensitive checks derive from; the persisted reward set is versioned and rejects unrecognized versions; the per-cycle sBTC recipient address is constructed so that the committed Bitcoin can move only through the intended sBTC deposit path; and the previous proof-of-transfer version is cleanly deprecated at the epoch boundary.

The one finding with a direct economic impact concerns the accounting for rewards in the new sBTC-backed protocol bonds. The bond-rollover path omits the share-removal step that every other bond-exit path performs, which lets a staker roll an ending bond forward, withdraw the collateral from the new bond, and still collect the old bond's final-cycle reward without committing collateral for that cycle; the excess reward is drawn from the reserve and the other stakers (TOB-STACKS-1).

A recurring pattern across the remaining findings is that the migration from the classic model (the mod-1 cycle boundary and two-output proof-of-transfer commits) to the

waterfall model (the mod-0 boundary and a single sBTC output) was not applied uniformly to every code path. A reward-cycle-boundary log field still reports the pre-waterfall boundary (TOB-STACKS-2), and the off-chain miner-spend estimators still model competing block-commits with the pre-waterfall output scheme (TOB-STACKS-3). These issues affect observability and off-chain tooling, but do not affect consensus; they indicate that the new model has not been propagated to all boundary-sensitive code. The two remaining findings are robustness issues with limited impact: a signer-signature ordering check validates only the first signer index, which permits block-header malleability but no signing-weight or threshold bypass (TOB-STACKS-4), and a configuration resolver honors a bond-admin override on mainnet, relying on a single configuration-layer guard rather than the two guards that protect the analogous sBTC values (TOB-STACKS-5).

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Stacks Labs take the following steps before activating the PoX-5 changes:

- **Remediate the findings disclosed in this report.** Only one finding (TOB-STACKS-1) carries a direct economic impact on the reward accounting of the new sBTC-backed bonds; the remaining findings are informational. These findings should be addressed through direct fixes or broader refactoring efforts.
- **Complete the propagation of the waterfall model across all boundary-sensitive code and tooling.** Audit every reward-cycle-boundary check and every off-chain estimator to confirm they use the epoch-aware waterfall model, so that logging and miner tooling remain consistent with consensus.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	0
Medium	0
Low	1
Informational	4
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Auditing and Logging	1
Configuration	1
Data Validation	3

Project Goals

The engagement was scoped to provide a security assessment of the Stacks Labs PoX-5 waterfall integration. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can a signer censor other signers from a sortition?
- Can a participant forge, reorder, duplicate, or replay signer signatures to get a block accepted that the honest signer set did not approve?
- Can a signer inflate their stake, signer-set weight, or rewards by manipulating the `pox-5` contract?
- Can a miner build a malicious block by bypassing singers' validations, or get honest nodes to accept it despite having an invalid transition?
- Can a miner halt the chain by manipulating the Stacks block header or the Bitcoin commit transaction data?
- Can a miner commit their BTC to an address other than the `pox-5` sBTC address for the reward cycle?
- Can the `pox-5` sBTC address lose access to the miner committed BTC?
- Can signers or stakers spend or prematurely unlock their locked STX tokens while still earning rewards or retaining signing weight?
- Can a single node influence or disrupt the canonical chain by modifying or misconfiguring consensus parameters, rather than merely forking itself off the network?
- Can a malicious staker steal funds from the `pox-5` contract?
- Are the `pox-5` reward calculations vulnerable to issues related to precision loss or other arithmetic considerations?
- Can a malicious staker spoof L1 BTC proofs or extract more rewards than expected?
- Is the `pox-5` smart contract integration with the node implemented correctly and resilient to issues related to data validation or access control bypasses?

Project Targets

The engagement involved reviewing and testing the following targets.

Stacks Core

Repository	https://github.com/stacks-network/stacks-core
Version	`4a99857f4297826e8d2123799f0c5e2b0fab3344`
Type	Smart contracts
Platform	Clarity

Stacks Core

Repository	https://github.com/stacks-network/stacks-core
Version	`aa85d3313ffa80010bf56df3305e10b8208e5163`
Type	Node client
Platform	Rust

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Epoch 4.0 activation and gating.** We reviewed the epoch schedule, the activation guards that constrain where PoX-5 may activate, and the configuration wiring that ties the PoX-5 activation height to the Epoch 4.0 start. We checked that the waterfall reward-cycle boundary is derived from a single source used by all waterfall-sensitive checks, and that the sBTC contract and bond-admin identities cannot be overridden on the mainnet.

- **Waterfall block-commits and sortition.** We reviewed the single-output block-commit format and the `check_pox_waterfall` validation and checked that every block in a waterfall cycle pays to the cycle's derived sBTC address, with no burn fallback. We assessed the impact of removing the anchor-block descendant check. We also reviewed the changes to the burn distribution and sortition weight computation implementation.

- **Reward set, signer set, and sBTC recipient derivation.** We reviewed the prepare-phase computation of the reward and signer sets and their compute-store-read-back lifecycle, the weight apportionment and its determinism, and whether a signer in the set can be silently excluded. We reviewed the derivation of the per-cycle sBTC P2TR deposit address, including the disabled key path and reclaim paths, and the byte-exact reproducibility required for consensus and the sBTC peg.

- **PoX-5 boot contracts.** We reviewed the deprecation of PoX-4 mutations in Epoch 4.0, and the one-time Epoch 4.0 boot deployment of the `pox-5` contract. We reviewed the contract's signer registration, the reentrancy-protected staking flow, and the staking, bond, and reward-accounting logic. Most importantly, we validated that the reward calculations, deposit, and withdrawal logic were sound to prevent a malicious staker from spoofing deposits or receiving more rewards than expected.

- **Native STX locking.** We reviewed the native lock, unstake, update, and rollover operations and their arithmetic (value conservation and checked overflow/underflow), the lazy-unlock read path, the deprecation of PoX-4 lock mutations, the forced auto-unlock of PoX-4 balances at Epoch 4.0, and the trust boundary in which the PoX-5 contract gates and the native hook apply the returned result.

- **Block validation, finality, and mining.** We reviewed the verification of signer signatures against the PoX-5 signer set and its stake-weighted approval threshold,

including the signature-ordering check. We reviewed the persistence of the reward set, the treatment-bitvector length handling for waterfall cycles, and the miner and relayer logic that builds the block-commit's committed outputs

- **Signer client.** We reviewed the changes to the standalone signer and the shared signer library to check that the signer correctly reads the PoX-5 signer set from the node and that the reward-set accessor changes are compatible with the waterfall variant.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- **Clarity 6 virtual machine changes.** The new Clarity 6 language features that the `pox-5` contract relies on, including the new on-chain Bitcoin built-in functions, the Ed25519 primitives, the cost functions, and the associated type-checker and analysis changes, were not reviewed as part of this engagement.
- **RPC, event, and networking surfaces.** The RPC endpoints, event-dispatch payloads, and peer-to-peer networking changes were not reviewed.
- **Consensus snapshot tests.** The `insta` consensus snapshot diffs, which serve as the project's guardrail against unintended consensus changes, were not reviewed.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Bond rollover lets a staker reclaim collateral while still earning the old bond's final-cycle reward	Data Validation	**Low**
2	Reward cycle start log field uses the pre-waterfall predicate	Auditing and Logging	**Informational**
3	Miner-spend estimators model block-commits with the pre-waterfall output scheme	Data Validation	**Informational**
4	Nakamoto block signature ordering check only validates the first signer index	Data Validation	**Informational**
5	PoX-5 bond-admin resolver allows the config override on the mainnet	Configuration	**Informational**

Detailed Findings

1. Bond rollover lets a staker reclaim collateral while still earning the old bond's final-cycle reward

Severity: **Low**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-STACKS-1
Target: `stackslib/src/chainstate/stacks/boot/pox-5.clar`	

Description

A bond participant earns a reward cycle only if their collateral is committed for that cycle, and a plain early exit reflects this: calling `unstake-sbtc` during a bond's final cycle removes that cycle's shares before its reward is computed, so the staker forfeits the final cycle in exchange for early access to their collateral. Rolling over from an ending bond into the next one bypasses that forfeiture. The staker can roll bond `N` into bond `N+6`, withdraw the collateral from the new bond, and still collect bond `N`'s final-cycle reward, because the rollover never removes bond `N`'s shares and the subsequent withdrawal does not reach them.

Bond reward shares are stored per reward cycle in maps keyed by `(reward-cycle, bond-index, …)`. Every exit path removes a staker's shares for the current and remaining cycles through `remove-staker-from-bond-cycles`: `unstake-sbtc`, `update-bond-registration`, and `announce-l1-early-exit` all call it. The rollover path in `register-for-bond` does not; it only adds the new bond's shares, as shown in figure 1.1.

```
(map-set protocol-bond-memberships tx-sender {
    bond-index: bond-index,
    amount-ustx: amount-ustx,
    signer: signer,
    is-l1-lock: (is-ok btc-lockup),
    amount-sats: sats-total,
})
(map-set protocol-bonds-total-staked bond-index
    (+ current-total-staked sats-total)
)
(try! (add-staker-to-bond-cycles tx-sender signer bond-index first-reward-cycle
    BOND_LENGTH_CYCLES sats-total
))
```

Figure 1.1: The rollover path adds the new bond's shares with no removal of the old bond's shares. (stackslib/src/chainstate/stacks/boot/pox-5.clar#L756–L768).

The rollover occurs in the second half of bond N's final cycle, and bond N's final-cycle reward is computed by a later `calculate-rewards` run that reads the per-cycle shares shown in figure 1.2, which still credit the rolled-over staker.

```
(total-sats (get-total-shares-staked-for-cycle reward-cycle (some bond-index)))
```

Figure 1.2: Bond rewards are distributed over the per-cycle shares, which still credit a rolled-over staker. (stackslib/src/chainstate/stacks/boot/pox-5.clar#L2109).

After rolling over with an equal amount, the staker withdraws the collateral from the new bond. The `unstake-sbtc` removal is clamped to the new bond's cycles through `clamp(current-cycle, bond-start, bond-end)`, so for bond N+6 it touches only cycles at or after `c_N+12` and never reaches bond N's final cycle, as shown in figure 1.3.

```
(first-changed-reward-cycle (clamp current-cycle bond-start-cycle bond-end-cycle))
(num-cycles (- bond-end-cycle first-changed-reward-cycle))
```

Figure 1.3: `unstake-sbtc` only adjusts the current and later cycles of its own bond, so it cannot reach a rolled-from bond's final cycle. (stackslib/src/chainstate/stacks/boot/pox-5.clar#L1241-L1242).

The result violates the protocol invariant that `total-sbtc-staked` equals the sum of custodied sBTC-backed bond shares: for bond N's final cycle, the credited shares exceed `total-sbtc-staked` by the withdrawn amount, and the bond's reward is sized on the inflated share total, so it draws more from the shared reward pool than its committed collateral warrants. The excess is taken from the reserve, the STX-only stakers, and lower-priority bonds in the distribution waterfall. The impact is bounded to a single cycle, because the rollover window only opens in the old bond's final half cycle and earlier cycles have already been computed. An honest rollover that keeps or increases the collateral is not affected by this issue and correctly continues to earn the final cycle, because its collateral remains committed.

Exploit Scenario

Alice holds an sBTC-backed position in bond N with S sats of collateral and is on the allowlist for bond N+6. During bond N's final cycle, she calls `register-for-bond` for bond N+6 with the same S sats, which is a no-op for `roll-sbtc` and leaves her bond-N final-cycle shares in place. She then calls `unstake-sbtc` against the new bond, which refunds her full S sats and sets `total-sbtc-staked` to zero without altering bond N's final-cycle shares. When `calculate-rewards` runs for that cycle, bond N is still active and distributes its reward over shares that still credit Alice, so she collects bond N's final-cycle reward with no collateral committed. A staker who instead exited bond N directly with `unstake-sbtc`

would have forfeited that cycle, so Alice has bypassed the early-exit forfeiture and shifted a cycle of bond rewards away from the reserve and the other stakers.

Recommendations

Short term, do not remove the old bond's shares on rollover; that would forfeit the final-cycle reward for honest rollovers, since the final cycle's `calculate-rewards` normally runs after the rollover window. Instead, keep the rolled-over collateral committed until the old bond ends: gate `unstake-sbtc` and the `roll-sbtc` refund so that a staker's custodied sBTC cannot drop below the old bond's outstanding share amount until `current-cycle` reaches the old bond's end cycle, and on a reducing rollover lower the old bond's remaining-cycle shares to the new amount. This prevents the issue because the final-cycle reward becomes payable only while the backing collateral is still committed, while an equal or increased rollover is unaffected.

Long term, add an invariant test asserting that a bond's reward for a cycle is earned only if collateral remained committed for that cycle, covering equal, increased, reduced, and fully withdrawn rollovers, and asserting that `total-sbtc-staked` equals the sum of custodied sBTC-backed bond shares after every rollover and unstake. This prevents the issue because it will detect any path that lets shares earn rewards without matching custodied collateral.

2. Reward cycle start log field uses the pre-waterfall predicate

Severity: **Informational**	Difficulty: **Undetermined**
Type: Auditing and Logging	Finding ID: TOB-STACKS-2
Target: `stackslib/src/chainstate/nakamoto/coordinator/mod.rs`	

Description

The `is_rc_start` field emitted by the per-burn-block `info!` log in the `handle_new_nakamoto_burnchain_block` function is computed from a different predicate than the one that actually determines whether the burn block begins a reward cycle. The log field calls the `self.burnchain.is_reward_cycle_start(header.block_height)` function, whereas the control-flow decision a few lines below calls the `is_naka_reward_cycle_start_for_epoch(&self.burnchain, header.block_height)` function.

```
info!(
    "Process burn block {} reward cycle {} in {}",
    header.block_height, reward_cycle, &self.burnchain.working_dir;
    "in_prepare_phase" => self.burnchain.is_in_prepare_phase(header.block_height),
    "is_rc_start" => self.burnchain.is_reward_cycle_start(header.block_height),
    "is_prior_in_prepare_phase" =>
self.burnchain.is_in_prepare_phase(header.block_height.saturating_sub(2)),
    "burn_block_hash" => %header.block_hash,
);
...
let is_reward_cycle_start =
    is_naka_reward_cycle_start_for_epoch(&self.burnchain, header.block_height);
let reward_cycle_info = if is_reward_cycle_start {
```

Figure 2.1: The `is_rc_start` log field and the actual reward-cycle-start decision use different predicates. (`stackslib/src/chainstate/nakamoto/coordinator/mod.rs`)

The two predicates do not agree in the PoX waterfall era. `is_naka_reward_cycle_start_for_epoch` returns the mod-0 boundary for burn heights at or after the first waterfall block, and the mod-1 boundary before it.

```
fn is_naka_reward_cycle_start_for_epoch(burnchain: &Burnchain, block_height: u64) ->
bool {
    let first_wf_block = burnchain
        .pox_constants
        .first_pox_waterfall_block(burnchain.first_block_height)
        .unwrap_or(u64::MAX);
```

```
    if block_height >= first_wf_block {
        burnchain.is_naka_signing_cycle_start(block_height)
    } else {
        burnchain.is_reward_cycle_start(block_height)
    }
}
```

Figure 2.2: `is_naka_reward_cycle_start_for_epoch` selects the mod-0 boundary in the waterfall era. (`stackslib/src/chainstate/nakamoto/coordinator/mod.rs`)

Before the first waterfall block, both predicates reduce to the mod-1 check, so the log field is accurate. From the first waterfall block onward, the decision uses the mod-0 check while the log continues to report the mod-1 check.

The impact is limited to observability: logs that describe reward-cycle boundaries in the waterfall era are misleading. The inconsistency also indicates that the migration from the mod-1 boundary to the mod-0 boundary may not have been applied uniformly across all reward-cycle-boundary-sensitive code paths.

Recommendations

Short term, change the `is_rc_start` log field to report the value of `is_naka_reward_cycle_start_for_epoch(&self.burnchain, header.block_height)`.

Long term, audit every reward-cycle-boundary check to confirm that it uses the epoch-aware predicate (the mod-0 boundary in the waterfall era) wherever the waterfall cycle boundary is relevant. Add tests asserting that the logged boundary, the boundary used to compute reward cycle information, and the boundary used by the prepare-phase reward-set computation all agree across the epoch-3.x-to-epoch-4.0 transition.

3. Miner-spend estimators model block-commits with the pre-waterfall output scheme

Severity: **Informational**	Difficulty: **Undetermined**
Type: Data Validation	Finding ID: TOB-STACKS-3
Target: `stackslib/src/config/chain_data.rs`, `stacks-node/src/main.rs`	

Description

The miner-spend estimators reconstruct competing miners' block-commits using the classic (pre-waterfall) PoX model: they call the classic `is_in_prepare_phase` predicate to decide the burn-distribution window and the expected number of commit outputs, and they treat a reward-phase commit as a two-output PoX commit. Under the PoX-5 waterfall (Epoch 4.0+), every block-commit in a reward cycle, including prepare-phase commits and the mod-0 cycle-start block, must pay a single sBTC taproot output, and the reward set is a `RewardSet::Waterfall` rather than a `RewardSet::V0`. The estimators do not account for either change, so the burn distribution and the derived spend recommendation they produce are computed against a stale model on a waterfall-era chain.

The `get_miner_spend` CLI helper performs the analogous computation without the waterfall arm. On a waterfall chain, it emits a burn output for any prepare-phase height (including the mod-0 cycle-start block, which under the waterfall must pay the sBTC address), and a generic PoX output set otherwise:

```
let commit_outs = if !burnchain.is_in_prepare_phase(tip.block_height + 1) {
    RewardSetInfo::into_commit_outs(recipients, config.is_mainnet())
} else {
    vec![PoxAddress::standard_burn_address(config.is_mainnet())]
};
```

Figure 3.2: `cli_get_miner_spend` builds commit outputs with no waterfall branch. (`stacks-node/src/main.rs`)

Similarly, the `MinerStats` reconstructs the competing-commit burn distribution using the classic predicate as well: it gates the multi-block commitment window on `is_in_prepare_phase`, sets the per-slot burn flags with it, and selects the window length from it. It also fixes the expected chained-UTXO input index from the same predicate. None of these branches use the single-output waterfall commit.

```
if !burnchain.is_in_prepare_phase(burn_block_height) {
    // PoX reward-phase is active!
```

```
    ...
    for blocks_back in 0..(MINING_COMMITMENT_WINDOW - 1) { ... }
} else {
    // PoX reward-phase is not active
    ...
    assert_eq!(windowed_block_commits.len(), 1);
}
```

Figure 3.3: `MinerStats::get_burn_distribution` windows competing commits with the classic predicate and the pre-waterfall output model. (`stackslib/src/config/chain_data.rs`)

Recommendations

Short term, update both estimators to use the single sBTC output for every block of a waterfall cycle.

Long term, add tests that assert the estimator's modeled commit outputs match the relayer's `commit_outs` for the same reward-set input in both the classic and waterfall regimes.

4. Nakamoto block signature ordering check only validates the first signer index

Severity: **Informational**	Difficulty: **Undetermined**
Type: Data Validation	Finding ID: TOB-STACKS-4
Target: `stackslib/src/chainstate/nakamoto/mod.rs`	

Description

The Nakamoto signer-signature ordering check updates its tracking index only for the first signature, so a non-adjacent out-of-order signature sequence is accepted. The `last_index` variable is initialized to `None` and is assigned only inside the `else` branch, which is reached exclusively for the first signature; on every subsequent iteration, the if `let Some(index)` branch is taken, and `last_index` is never reassigned.

```
// `last_index` is used to prevent out-of-order signatures
let mut last_index = None;
```

Figure 4.1: last_index is initialized to None (stackslib/src/chainstate/nakamoto/mod.rs:861-862)

```
if let Some(index) = last_index.as_ref() {
    if *index >= signer_index {
        return Err(ChainstateError::InvalidStacksBlock(
            "Signatures are out of order".to_string(),
        ));
    }
} else {
    last_index = Some(signer_index);
}
```

Figure 4.2: last_index is assigned only on the first signature and never updated thereafter. (stackslib/src/chainstate/nakamoto/mod.rs:901-909)

Every signature after the first is compared against the index of the first signature alone, so a sequence such as [0, 2, 1] passes the ascending-order check even though it is not sorted. The remaining safeguards limit the impact: deduplication is consume-once via `signers_by_pk.remove(...)`, so the same signer cannot be counted twice; weight cannot be inflated; and signatures from keys outside the reward set are still rejected.

Because signer signatures are excluded from both `signer_signature_hash_inner()` and `block_hash()`, two byte-different serialized headers can resolve to the same block identifier, so the effect is limited to header malleability and loss of a canonical serialization

rather than any weight or threshold bypass. The existing test `test_out_of_order_signer_signatures` exercises only a fully reversed sequence [2, 1, 0], which trips the first-signature comparison and therefore does not catch the [0, 2, 1] case.

Exploit Scenario

An adversary, relaying a validly signed Nakamoto block, reorders the collected signer signatures into a non-monotonic but non-adjacent sequence, such as [0, 2, 1]. The reordered header passes signature verification and is accepted, producing a serialized block that differs byte-for-byte from the canonical form yet maps to the same block identifier, which can complicate caching, equality checks, and gossip-layer assumptions that rely on a single canonical serialization.

Recommendations

Short term, update `last_index` on every iteration of the signature loop so each signer index is compared against the immediately preceding accepted index.

Long term, extend `test_out_of_order_signer_signatures` to cover non-adjacent permutations such as [0, 2, 1] in addition to the fully reversed case. This prevents regressions because the test suite will then exercise the ordering check beyond the first index.

5. PoX-5 bond-admin resolver allows the config override on the mainnet

Severity: **Informational**	Difficulty: **Undetermined**
Type: Configuration	Finding ID: TOB-STACKS-5
Target: `stackslib/src/chainstate/nakamoto/signer_set.rs`	

Description

The `pox_5_bond_admin` function checks the process-global override before checking `is_mainnet`, so a configured override is honored on mainnet. Its sibling resolvers, `pox_5_sbtc_contract` and `pox_5_sbtc_registry_contract`, return the canonical mainnet identifier first and only then check the override off-mainnet.

```
pub fn pox_5_bond_admin(is_mainnet: bool) -> PrincipalData {
    let principal = POX_5_BOND_ADMIN.read().unwrap().clone();
    if let Some(principal) = principal {        // override wins, even on mainnet
        principal
    } else if is_mainnet {
        PrincipalData::parse(POX_5_BOND_ADMIN_MAINNET) ...
    } else { ... }
}
```

Figure 5.1: The bond-admin resolver checks the override before `is_mainnet`. (`stackslib/src/chainstate/nakamoto/signer_set.rs`)

All three values are substituted into the deployed `pox-5` contract body by `make_pox_5_body`, so each is consensus-critical: a node that resolves a different value deploys a divergent `pox-5` contract. The bond-admin doc comment states that the "Forbidden on mainnet" invariant is enforced for the bond admin only by the `Config` validation that rejects `pox_5_bond_admin` on mainnet, whereas the sBTC values are protected at both the config layer and the resolver.

The impact is limited because a divergent value would only self-fork the node rather than affect the network. However, `set_pox_5_bond_admin` is `pub` and is already invoked directly by tests, so the mainnet invariant depends on no caller ever bypassing the config check.

Exploit Scenario

A later change adds a second path that sets the bond-admin override (e.g., a new config route, an embedder, a refactor that relocates the mainnet reject, or a test/tooling helper linked into a release build) and runs it on a mainnet node. `make_pox_5_body` bakes the override into the deployed `pox-5` contract, diverging that node's chainstate.

Recommendations

Short term, short-circuit `pox_5_bond_admin` on `is_mainnet` to return `POX_5_BOND_ADMIN_MAINNET` before reading the override.

Long term, route the three `set_pox_5_*` overrides through a single startup entry point (or gate them behind a test/debug flag) so the mainnet invariant cannot be re-introduced by a stray caller.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

The following recommendations are not associated with immediate security issues, but they may warrant further investigation.

- **Document `set-burnchain-parameters` frontrunning protection.**
  Anyone can call `set-burnchain-parameters` after deployment. The client has indicated that the function is called by consensus immediately after deployment; therefore, frontrunning would be impossible.

- **The `setup-bond` function does not need to have the `bond-index` as an input parameter.**
  Because bonds cannot be created "too far in the future", the `bond-index` parameter could be removed and instead rely on an internal counter to determine the index.

- **Replace `unwrap-panic` usage with `unwrap!` or `try!` for clearer error handling.**

- **Include the burnchain parameters that are input into the `set-burnchain-parameters` function as part of the return, instead of simply returning (`ok true`).**

C. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not a comprehensive analysis of the system.

From July 6 to July 7, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Stacks Labs team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the five issues described in this report, Stacks Labs has resolved three issues and has not resolved the remaining two issues. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Bond rollover lets a staker reclaim collateral while still earning the old bond's final-cycle reward	Low	Unresolved
2	Reward cycle start log field uses the pre-waterfall predicate	Informational	Resolved
3	Miner-spend estimators model block-commits with the pre-waterfall output scheme	Informational	Unresolved
4	Nakamoto block signature ordering check only validates the first signer index	Informational	Resolved
5	PoX-5 bond-admin resolver allows the config override on the mainnet	Informational	Resolved

Detailed Fix Review Results

TOB-STACKS-1: Bond rollover lets a staker reclaim collateral while still earning the old bond's final-cycle reward
Unresolved. The client provided the following context for this finding's fix status:

> *This is intended behavior and will not be fixed.*

TOB-STACKS-2: Reward cycle start log field uses the pre-waterfall predicate
Resolved in PR #7392. The log statements now use the correct reward cycle start block.

TOB-STACKS-3: Miner-spend estimators model block-commits with the pre-waterfall output scheme
Unresolved. The client provided the following context for this finding's fix status:

> *This one is still unfixed, but it only effects a CLI that we believe is mostly unused at the moment. Regardless, it won't be fixed today*

TOB-STACKS-4: Nakamoto block signature ordering check only validates the first signer index
Resolved in PR #7016. The block signature validation loop now updates the last signature index in every iteration for epoch 4.0 blocks.

TOB-STACKS-5: PoX-5 bond-admin resolver allows the config override on the mainnet
Resolved in PR #7360. The `pox_5_bond_admin` function has been updated to not allow config overrides on the mainnet.

D. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Stacks Labs under the terms of the project statement of work and has been made public at Stacks Labs' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.